1. Explain the concept of type conversion in Python. Differentiate between implicit and explicit conversion with examples.

Type conversion is the process of changing the data type of a variable from one type to another. Python often needs this when performing operations between different data types such as **integers, floats, and strings**.

Example:
If we add an integer and a float, Python converts the integer into a float so that the operation can be performed correctly.

Type conversion in Python occurs in **two ways**:

1. **Implicit Type Conversion (Automatic)**
2. **Explicit Type Conversion (Manual / Type Casting)**

---

1. Implicit Type Conversion

Implicit conversion is **automatically performed by Python**.
Python converts a smaller data type into a larger data type to avoid data loss.

Example

```
a = 10       # integer
b = 5.5      # float

c = a + b    # Python converts a to float automatically

print(c)
print(type(c))
```

Output

```
15.5
<class 'float'>
```

Explanation:

- a is an **int**
- b is a **float**
- Python converts a → 10.0 automatically
- Result becomes **float**

This automatic conversion is called **implicit type conversion**.

2. Explicit Type Conversion (Type Casting)

Explicit conversion is **manually done by the programmer** using built-in functions.

Common type conversion functions in Python:

Function	Purpose
int()	Converts value to integer
float()	Converts value to float
str()	Converts value to string
bool()	Converts value to boolean

Example

```
a = "20"
b = 10

c = int(a) + b   # manually converting string to integer

print(c)
```

Output

30

Explanation:

- a is a **string**
- We convert it using int(a)
- Then addition is performed

2. Develop a Python program with a while loop to display the Fibonacci sequence up to n terms entered by the user.

The **Fibonacci sequence** is a series of numbers where each number is the **sum of the two preceding numbers**.
The sequence usually starts with **0 and 1**.

Example:
0, 1, 1, 2, 3, 5, 8, 13, ...

Python Program

```
# Fibonacci sequence using while loop

n = int(input("Enter the number of terms: "))

a = 0
b = 1
count = 0

if n <= 0:
   print("Please enter a positive number")
else:
   print("Fibonacci sequence:")

   while count < n:
      print(a, end=" ")
      c = a + b
      a = b
      b = c
      count += 1
```

Explanation

1. The user enters the number of terms **n**.
2. The first two Fibonacci numbers are stored in variables **a = 0** and **b = 1**.
3. A **while loop** runs until the count reaches **n**.
4. Each new term is calculated using

   c = a + b

5. The values are updated for the next iteration.

Example Output

Enter the number of terms: 7
Fibonacci sequence:
0 1 1 2 3 5 8

3. Differentiate between a syntax error and a runtime error with examples.

Difference Between Syntax Error and Runtime Error

Feature	**Syntax Error**	**Runtime Error**
Definition	Error due to incorrect syntax	Error occurring during execution
Detection	Detected before program execution	Detected while program runs
Cause	Missing symbols, incorrect indentation, wrong statements	Invalid operations like division by zero
Program Execution	Program does not start	Program starts but stops when error occurs
Example	print("Hello"	10/0

4. Describe the Collatz 3n + 1 sequence and explain how iteration and conditional

Collatz 3n + 1 Sequence

The **Collatz sequence** (also called the **3n + 1 sequence**) is a mathematical process applied to a positive integer. It generates a sequence of numbers using simple rules until the number becomes **1**.

Rules of the Collatz Sequence

1. Start with any **positive integer n**.
2. If **n is even**, divide it by **2**.

   n=n2n = \frac{n}{2}n=2n

3. If **n is odd**, multiply it by **3 and add 1**.

   n=3n+1n = 3n + 1n=3n+1

4. Repeat the process until **n becomes 1**.

This process is known as the **Collatz Conjecture**, which suggests that **every positive integer eventually reaches 1**, though it has not been mathematically proven for all numbers.

Example

Start with **n = 6**

Sequence generated:

6 → 3 → 10 → 5 → 16 → 8 → 4 → 2 → 1

Explanation:

- 6 is even → 6 / 2 = 3
- 3 is odd → 3 × 3 + 1 = 10
- 10 is even → 10 / 2 = 5
- Continue until the value becomes **1**

Use of Iteration and Conditional Statements

1. Iteration

Iteration means **repeating a set of instructions** until a condition is satisfied.
In the Collatz sequence, a loop (usually a **while loop**) repeatedly applies the rules until **n becomes 1**.

2. Conditional Statements

Conditional statements (**if–else**) are used to **check whether the number is even or odd** and apply the correct rule.

- If the number is **even**, divide by 2.
- If the number is **odd**, apply **3n + 1**.

Python Example

```
n = int(input("Enter a positive integer: "))

while n != 1:
    print(n, end=" ")

    if n % 2 == 0:
        n = n // 2
    else:
```

```
    n = 3 * n + 1

print(1)
```

Explanation of the Program

- The **while loop** performs the iteration until n becomes **1**.
- The **if–else condition** checks whether the number is **even or odd**.
- The corresponding rule (**n/2** or **3n + 1**) is applied.

5. Develop a program that prints all numbers from 1 to 100 that are divisible by 3 or 5 but not both. Use continue or break statements wherever suitable.

The program should print numbers between **1 and 100** that are **divisible by 3 or 5 but not both**. This condition means:

- Number divisible by **3 only**, or
- Number divisible by **5 only**,
- But **not divisible by both 3 and 5**.

The **continue statement** is used to skip numbers divisible by both.

Program

```
for i in range(1, 101):

    # Skip numbers divisible by both 3 and 5
    if i % 3 == 0 and i % 5 == 0:
        continue

    # Print numbers divisible by 3 or 5
    if i % 3 == 0 or i % 5 == 0:
        print(i, end=" ")
```

Explanation

1. range(1, 101) generates numbers from **1 to 100**.
2. if i % 3 == 0 and i % 5 == 0: checks numbers divisible by **both 3 and 5**.
3. continue skips those numbers.
4. The second condition prints numbers divisible by **3 or 5 only**.

Sample Output (Partial)

3 5 6 9 10 12 18 20 21 24 25 ...

6. What is meant by function composition? Illustrate with an example.

Function Composition

Function composition means **combining two or more functions so that the output of one function becomes the input of another function**.
It allows complex operations to be built from simpler functions.

In mathematics, if we have two functions **f(x)** and **g(x)**, the composition is written as:

$(f \circ g)(x)=f(g(x))$

This means **first apply function g to x, then apply function f to the result**.

Example in Mathematics

Let

- $f(x)=x+2$
- $g(x)=2x$

Then

$(f \circ g)(x) = f(g(x))$

Step-1:
$g(x))=2x$

Step-2:
$f(g(x))= f(2x)=2x+2$

Example in Python

```
def g(x):
    return 2 * x

def f(x):
    return x + 2

x = 5
result = f(g(x))   # composition of functions
```

```
print("Result:", result)
```

Output

Result: 12

Explanation

- g(5) → 2×5=102 × 5 = 102×5=10
- f(10) → 10+2=1210 + 2 = 1210+2=12

So the composed function **f(g(5)) = 12**.

7.Explain the string operations in Python for slicing, concatenation, repetition, and Comparison with suitable examples.

String Operations in Python

A **string** in Python is a sequence of characters enclosed in **single quotes (' ')**, **double quotes (" ")**, or **triple quotes**. Python provides several operations to manipulate strings such as **slicing, concatenation, repetition, and comparison**.

---

1. String Slicing

Slicing is used to extract a part of a string using index positions.

Syntax

```
string[start : end]
```

- start → starting index
- end → ending index (not included)

Example

```
s = "Python Programming"

print(s[0:6])   # first 6 characters
print(s[7:18])  # substring
```

Output

Python
Programming

Explanation

- s[0:6] returns characters from index **0 to 5**.
- The **end index is excluded**.

---

2. String Concatenation

Concatenation means **joining two or more strings** using the + operator.

Example

```
s1 = "Hello"
s2 = "World"

s3 = s1 + " " + s2
print(s3)
```

Output

Hello World

Explanation

- The + operator joins s1 and s2 to form a single string.

---

3. String Repetition

Repetition repeats a string multiple times using the * operator.

Example

```
s = "Python "
print(s * 3)
```

Output

Python Python Python

Explanation

- The * operator repeats the string **3 times**.

---

4. String Comparison

Strings can be compared using **comparison operators** such as:
==, !=, <, >, <=, >=.

Python compares strings **character by character based on ASCII values**.

Example

```
s1 = "apple"
s2 = "banana"

print(s1 == s2)
print(s1 < s2)
print(s1 > s2)
```

Output

```
False
True
False
```

Explanation

- "apple" < "banana" because the ASCII value of **'a'** is smaller than **'b'**.

8. Define a list. How is it different from an array? Develop a Python statement to access the third element of a list: nums = [3, 6, 9, 12].

A **list** in Python is an **ordered collection of elements** that can store items of **different data types** such as integers, strings, floats, or even other lists. Lists are **mutable**, which means their elements can be **modified after creation**.

Example

```
nums = [3, 6, 9, 12]
```

Here, nums is a list containing four elements.

Difference Between List and Array

Feature	List	Array
Definition	Collection of elements that can store different data types	Collection of elements of the **same data type**
Data Types	Can store **mixed data types**	Stores **only one type** (e.g., all integers)
Flexibility	More flexible and commonly used in Python	Less flexible
Library Requirement	Built-in in Python	Usually requires modules like array or numpy
Example	[1, "Python", 3.5]	[1, 2, 3, 4]

Python Statement to Access the Third Element

Python uses **indexing starting from 0**.

For the list:

nums = [3, 6, 9, 12]

Index positions:

3 → index 0
6 → index 1
9 → index 2
12 → index 3

Statement

```
print(nums[2])
```

Output

9

9. Develop a program to count the number of words in a given line of text.

A **word count program** counts how many words are present in a given sentence or line of text entered by the user.

Program

```
# Program to count number of words in a line of text

text = input("Enter a line of text: ")

words = text.split()   # split sentence into words
count = len(words)     # count number of words

print("Number of words:", count)
```

Explanation

1. input() is used to read a line of text from the user.
2. split() divides the sentence into a list of words using spaces as separators.
3. len() counts the number of elements (words) in the list.
4. The result is printed.

Example Output

```
Enter a line of text: Python is easy to learn
Number of words: 5
```

10. Explain mutability in lists. Illustrate the difference between modifying a list and Creating a clone of it using examples.

Mutability in Lists

Mutability means that the **contents of an object can be changed after it is created**.
In Python, **lists are mutable**, which means we can **add, remove, or modify elements** without creating a new list.

Example:

```
nums = [1, 2, 3, 4]
nums[2] = 10
print(nums)
```

Output

```
[1, 2, 10, 4]
```

Here, the element at index **2** is changed from **3 to 10**, showing that lists are mutable.

---

1. Modifying a List (Aliasing)

When we assign one list to another variable, both variables **refer to the same list object**.
If we modify one list, the other will also change.

Example

```
list1 = [10, 20, 30]
list2 = list1     # aliasing

list2[0] = 100

print("list1:", list1)
print("list2:", list2)
```

Output

```
list1: [100, 20, 30]
list2: [100, 20, 30]
```

Explanation

- list1 and list2 refer to the **same memory location**.
- Changing list2 also changes list1.

---

2. Creating a Clone of a List

A **clone** means creating a **separate copy of the list**.
Changes made to the cloned list **do not affect the original list**.

Example

```
list1 = [10, 20, 30]
list2 = list1[:]    # cloning the list

list2[0] = 100

print("list1:", list1)
print("list2:", list2)
```

Output

```
list1: [10, 20, 30]
list2: [100, 20, 30]
```

Explanation

- list2 = list1[:] creates a **new copy of the list**.
- Modifying list2 does **not change** list1.

Difference Between Modifying and Cloning a List

Feature	Modifying (Aliasing)	Cloning
Memory Location	Same memory location	Different memory location
Effect of Change	Change affects both lists	Change affects only the cloned list
Method	list2 = list1	list2 = list1[:] or list(list1)
Example Result	Both lists change	Only cloned list changes

11. Develop a Python program to check if a string is a palindrome using slicing.

A **palindrome** is a word, phrase, or number that reads the **same forward and backward**.

Examples:

- madam
- level
- radar

In Python, we can use **string slicing** to reverse the string and compare it with the original string.

Program

```
# Program to check if a string is a palindrome

text = input("Enter a string: ")

# Reverse the string using slicing
reverse_text = text[::-1]

if text == reverse_text:
    print("The string is a palindrome")
else:
    print("The string is not a palindrome")
```

Explanation

1. input() reads the string from the user.
2. text[::-1] reverses the string using slicing.
3. The program compares the **original string** with the **reversed string**.
4. If both are equal, the string is a **palindrome**.

Example Output

Enter a string: madam
The string is a palindrome
Enter a string: python
The string is not a palindrome

12. Develop a program that takes a list of numbers and returns a new list containing only the even numbers.

This program takes a **list of numbers** and creates a **new list containing only the even numbers**.

Program

```
# Program to filter even numbers from a list

numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_numbers = []

for num in numbers:
    if num % 2 == 0:
        even_numbers.append(num)

print("Original List:", numbers)
print("Even Numbers:", even_numbers)
```

Explanation

1. A list numbers is created containing several values.
2. An empty list even_numbers is created to store even numbers.
3. The **for loop** checks each number in the list.
4. num % 2 == 0 checks if the number is **even**.
5. If the condition is true, the number is added to the new list using append().